

INTRODUCTION TO POWER BI

Unit II – Power BI Desktop and Data Transformation

27 Model Answers | 5–6 Marks Each

Reference: Unit II PPT Slides & Detailed Study Notes

Unit II: Power BI Desktop and Data Transformation – All 27 Answers

Q1. Explain the role of Power BI Desktop in the data analysis process.

Introduction

Power BI Desktop is a free Windows application developed by Microsoft that serves as the primary tool for data analysis, transformation, modelling, and report creation. It is the starting point of the Power BI workflow, where raw data is converted into meaningful visual insights.

Core Roles of Power BI Desktop

- **Data Connection:** It connects to 200+ data sources including Excel, CSV, SQL Server, Azure, Google Analytics, REST APIs, and web pages using the 'Get Data' feature.
- **Data Transformation:** The built-in Power Query Editor allows users to clean, reshape, filter, and structure raw data before it is loaded into the model.
- **Data Modelling:** Users define relationships between tables, create hierarchies, and write DAX measures to build an optimised data model in the Model View.
- **Report Creation:** The Report View provides a drag-and-drop canvas where users place visualisations such as bar charts, pie charts, maps, and KPI cards to design interactive reports.
- **Publishing:** Completed .pbix reports are published to Power BI Service (cloud) for sharing, collaboration, and dashboard creation.

Three Main Views

View	Purpose	Key Feature
Report View	Design visuals and reports	Drag-and-drop canvas, Visualisations pane
Data View	Inspect and understand the loaded data	See column values, data types, calculated columns
Model View	Manage table relationships	Create, edit, and view relationships between tables

Key Insight (from PPT): 'Desktop is where the heavy lifting of transformation happens; the Service is where insights are consumed.'

Q2. Analyze the importance of data transformation before visualization in Power BI.

What is Data Transformation?

Data transformation is the process of converting raw, unstructured, or inconsistent data into a clean, structured format that is ready for analysis. In Power BI, this is done using the Power Query Editor before any visuals are created.

Why Transformation Must Come Before Visualization

- **Accuracy of Reports:** If data contains duplicates, null values, or incorrect data types, every chart built on it will produce wrong results. For example, a SUM of a text column returns zero instead of a total.

- **Correct DAX Calculations:** DAX functions like SUM(), AVERAGE(), and CALCULATE() depend on correct data types and clean fields. Dirty data directly causes calculation errors.
- **Consistent Formats:** Dates stored as text, inconsistent capitalisation (e.g., 'Mumbai' vs 'mumbai'), or mixed units (kg vs g) make group-by and filter operations unreliable.
- **Performance Optimisation:** Removing unnecessary columns, filtering irrelevant rows, and setting correct data types reduces the size of the in-memory model, resulting in faster report loading.
- **Reliable Relationships:** Table relationships can only be created on clean, unique key columns. Duplicate or null keys in a column prevent proper relationship creation.

Common Transformation Tasks (from PPT)

- **Change Data Types:** Convert text to number, number to date, etc.
- **Split / Merge Columns:** Separate full names into first/last; merge city and postal code.
- **Remove Duplicates and Nulls:** Eliminate repeated rows and fill or remove missing values.
- **Filter Rows:** Remove test records, blank rows, or out-of-scope data.
- **Pivot / Unpivot:** Restructure data from wide to tall format for analysis.

Conclusion: Transformation is the foundation of every reliable Power BI report. Visualising untransformed data is like building on sand — the output will be incorrect regardless of how good the visual design is.

Q3. Demonstrate the process of loading data from different sources into Power BI Desktop.

Overview

Power BI Desktop supports 200+ data connectors. Loading data is the first step in building any report and is performed through the 'Get Data' feature available on the Home ribbon.

Step-by-Step Process

1. **Open Power BI Desktop:** Launch the application and start a new report.
2. **Click 'Get Data':** On the Home ribbon, click the 'Get Data' button. A dialog box displays all available connector categories.
3. **Select the Data Source Type:** Choose the appropriate connector — e.g., Excel Workbook, SQL Server Database, Web, Text/CSV, Azure, or API.
4. **Provide Connection Details:** Enter server name, database name, URL, or file path. Supply credentials if required (username/password for SQL Server; API key for web services).
5. **Navigator Window:** The Navigator window appears, showing a preview of available tables and fields. Select only the tables required to keep the model lean and improve performance.
6. **Load or Transform:** Click 'Load' to import directly into the model, or 'Transform Data' to open the Power Query Editor for cleaning before loading.

Supported Data Sources (from PPT and Notes)

Source Type	Examples	Use Case
File-Based	Excel, CSV, JSON, XML	Static reports, historical data
Database	SQL Server, MySQL, Oracle	Transactional data, ERP systems
Cloud	Azure SQL, Salesforce, SharePoint	Live business data, CRM

Web / API	REST APIs, OData, Web pages	External feeds, real-time data
------------------	-----------------------------	--------------------------------

Navigator Tip (from PPT): 'Choosing only what is relevant keeps models lean, improves refresh times, and limits downstream complexity.'

Q4. Compare various desktop views available in Power BI and explain their significance.

Overview

Power BI Desktop provides three distinct views, each designed for a specific stage of the analytics workflow. Understanding the purpose of each view is essential for efficient report development.

Detailed Comparison of Three Views

Feature	Report View	Data View	Model View
Purpose	Create interactive reports and dashboards	Inspect and review loaded data tables	Define and manage table relationships
Primary Users	Report designers, business analysts	Data validators, analysts checking data quality	Data modellers, BI developers
Key Features	Canvas, Visualisations pane, Fields pane, Slicers	Table grid, column profiling, data type check	Relationship lines, cardinality, cross-filter direction
When to Use	Building and formatting visuals	Verifying data after transformation	Setting up joins between Fact and Dimension tables

Significance of Each View

- **Report View:** This is the primary output layer — every chart, KPI card, slicer, and table that users will interact with is created here. It directly determines the quality and usability of the final report.
- **Data View:** It allows analysts to verify that data has been loaded correctly. Incorrect data types, unexpected nulls, or failed transformations are visible here before reports are built.
- **Model View:** Incorrect or missing relationships between tables produce wrong calculations. The Model View is where the analytical correctness of the entire report is established through proper Star Schema design.

Key Takeaway (from PPT): 'Power BI Desktop is organised around three main views, each serving a different role in shaping data into insights.'

Q5. Explain how Query Editor helps in cleaning and transforming raw data.

What is Power Query Editor?

The Power Query Editor is an integrated data preparation tool within Power BI Desktop. It provides a structured, step-by-step workspace where users can clean, reshape, and enrich raw data before it is loaded into the report model. It is accessed via Home > Transform Data on the ribbon.

Key Cleaning and Transformation Capabilities

- **Change Data Types:** Converts columns to the correct data type (text to number, text to date). This is critical because DAX calculations fail if a numeric column is stored as text.
- **Remove Duplicates:** Eliminates repeated rows to ensure aggregations like COUNT and SUM are not inflated by duplicate entries.
- **Handle Null and Missing Values:** Users can replace nulls with a default value, remove rows with nulls, or use 'Fill Down' to populate missing cells with the value above.
- **Split Columns:** Divides a single column into multiple columns — e.g., splitting a 'Full Name' column into 'First Name' and 'Last Name'.
- **Merge Columns:** Combines two or more columns into one — e.g., merging 'City' and 'State' into a single 'Location' column.
- **Filter Rows:** Removes irrelevant, test, or out-of-scope records from the dataset.
- **Pivot / Unpivot:** Restructures wide-format tables (one column per month) into tall-format tables (one row per month) required for time-series analysis.
- **Replace Values:** Corrects data entry errors or standardises values — e.g., replacing 'Mum' with 'Mumbai' across an entire column.

Applied Steps Pane

Every action taken in the Power Query Editor is recorded as a step in the 'Applied Steps' pane on the right side. This provides a transparent, auditable, and reversible transformation log. Any step can be deleted or reordered without re-importing the data.

M Language

Underlying every transformation is the M language (Power Query Formula Language). Advanced users can write custom M code directly for transformations not available in the standard interface.

From PPT: 'The Power Query Editor is the staging area where raw inputs are shaped into analysis-ready data. Every decision made here determines whether reports run well or fail.'

Q6. Analyze the importance of shaping data for effective reporting in Power BI.

What is Data Shaping?

Data shaping refers to the process of restructuring, organising, and formatting data into the exact structure needed for analysis and reporting. It goes beyond basic cleaning — it is about making data 'report-ready' by aligning it to the expected analytical model.

Why Data Shaping is Important

- **Correct Report Structure:** Raw data exported from systems like CRM, ERP, or spreadsheets rarely matches the structure Power BI needs. Shaping ensures the data fits the required Star Schema (Fact and Dimension tables).
- **Enables Accurate Aggregations:** Unshaped data in wide format (e.g., separate columns for Jan, Feb, Mar sales) cannot be easily aggregated. Unpivoting this into rows allows proper SUM, AVERAGE, and time-series analysis.
- **Improves Visual Accuracy:** Improperly shaped data causes visuals to display incorrect categories, missing values, or misaligned axes, making the report misleading.
- **Simplifies DAX Calculations:** Well-shaped tables allow DAX functions to traverse relationships correctly. For example, CALCULATE() with a date dimension requires a single Date column, not 12 separate month columns.

- **Reusability:** A well-shaped data model can be reused across multiple reports without reshaping each time.

Common Shaping Operations (from PPT)

- **Transpose:** Converts rows to columns or vice versa for reorienting the entire table.
- **Unpivot Columns:** Normalises wide-format tables where multiple attribute columns are collapsed into key-value pairs.
- **Merge Queries:** Joins two tables on a common key, similar to a SQL JOIN, combining columns from both sources.
- **Append Queries:** Stacks rows from two tables with the same structure — similar to a SQL UNION.

Conclusion: Data shaping is not about changing the meaning of data — it is about presenting the same data in the format that Power BI's visualisation and DAX engine can consume accurately and efficiently.

Q7. Demonstrate the steps involved in transforming and cleaning inconsistent datasets.

Scenario

Consider a sales dataset exported from a CRM system containing: duplicate order entries, dates stored as text, product names with inconsistent casing, missing customer IDs, and a combined 'Full Name' column.

Step-by-Step Transformation and Cleaning Process

7. **Open Power Query Editor:** In Power BI Desktop, click Home > Transform Data to open the Power Query Editor.
8. **Fix Data Types:** Select the 'Date' column. Click the data type icon (shown as ABC or 123) and change it from Text to Date. Repeat for any numeric columns stored as text.
9. **Remove Duplicate Rows:** Select the 'Order ID' column > Right-click > Remove Duplicates. This ensures each order is counted only once in aggregations.
10. **Handle Missing Values:** Select the 'Customer ID' column > Home > Remove Rows > Remove Blank Rows. Alternatively, right-click the column > Replace Values > replace null with a default like 'Unknown'.
11. **Standardise Text Casing:** Select the 'Product Name' column > Transform tab > Format > UPPERCASE or Capitalize Each Word to ensure consistent grouping in charts.
12. **Split a Combined Column:** Select the 'Full Name' column > Transform > Split Column > By Delimiter (space). This creates 'Full Name.1' (First Name) and 'Full Name.2' (Last Name). Rename the columns accordingly.
13. **Replace Incorrect Values:** Select the 'City' column > Transform > Replace Values. Replace 'Mum' with 'Mumbai', 'Del' with 'Delhi', etc. to standardise location data.
14. **Filter Irrelevant Rows:** Click the dropdown arrow on the 'Status' column > Filter > deselect 'Test' or 'Cancelled' records that should not appear in reports.
15. **Verify Applied Steps:** Review the Applied Steps pane on the right. Each step is listed and can be renamed, reordered, or deleted if needed.
16. **Close and Apply:** Click Home > Close & Apply. The cleaned data is loaded into the Power BI model.

From Notes: 'Data cleaning ensures the dataset is accurate, consistent, and error-free — Remove duplicates, Handle missing values, Correct spelling errors, Standardise formats, Remove blank rows, Fix incorrect data types.'

Q8. Explain the process of managing data relationships in Power BI Desktop.

What are Data Relationships?

A data relationship in Power BI is a connection between two tables established through a common column (key). Relationships allow Power BI to filter and aggregate data across multiple tables, enabling cross-table analysis without duplicating data.

How to Create a Relationship

17. **Go to Model View:** Click the Model View icon on the left sidebar of Power BI Desktop.
18. **Auto-Detection:** Power BI automatically detects relationships when tables share columns with matching names and data types. These appear as connecting lines between tables.
19. **Manual Creation:** Drag and drop a column from one table onto a matching column in another table. Alternatively, click Manage Relationships > New.
20. **Configure Properties:** In the relationship dialog, set: Cardinality (One-to-Many, Many-to-Many, One-to-One) and Cross-filter Direction (Single or Both).
21. **Validate:** The relationship line appears between the two tables. A '1' and '*' symbol indicate the One side and the Many side respectively.

Relationship Properties

Property	Explanation
Cardinality	Defines the nature of the relationship: One-to-Many (most common), Many-to-Many, One-to-One.
Cross-filter Direction	Controls which direction filters flow between tables — Single (one direction) or Both (bidirectional).
Active / Inactive	Only one relationship between two tables can be active. Inactive relationships can be used in DAX via USERELATIONSHIP().

Managing Existing Relationships

- **Edit:** Double-click any relationship line to modify its cardinality or filter direction.
- **Delete:** Right-click the relationship line > Delete.
- **Manage Relationships Dialog:** Home > Manage Relationships provides a tabular list of all relationships for bulk management.

Key Point: Correct data relationships are the backbone of accurate DAX calculations. Without relationships, a measure like 'Total Sales by Region' cannot link the Sales table to the Region table.

Q9. Differentiate between single-direction and cross-filter direction in Power BI relationships.

Overview

Cross-filter direction determines how filter context flows between related tables when a user interacts with a report. Choosing the correct direction directly affects the accuracy and behaviour of visualisations.

Single Cross-Filter Direction

- **Definition:** Filters flow in one direction only — from the 'One' side (dimension table) to the 'Many' side (fact table).
- **Behaviour:** If a user selects 'Electronics' in a Product Category slicer, the Sales table is filtered to show only electronics sales. But selecting a value in the Sales table does NOT filter the Product table.
- **Use Case:** Standard Star Schema — Dimension tables filter the Fact table. This is the default and recommended setting for most reports.
- **Example:** Dim_Product (One) -> Fact_Sales (Many): selecting a product filters sales, but selecting a sale does not filter the product list.

Both (Bidirectional) Cross-Filter Direction

- **Definition:** Filters flow in both directions — the Fact table can also filter the Dimension table.
- **Behaviour:** Selecting a sales record can filter the Product table to show only products involved in that sale, and vice versa.
- **Use Case:** Many-to-many relationships, role-playing dimensions, or when slicers in reports need to show only relevant dimension values based on available data.
- **Example:** A customer slicer showing only customers who have placed at least one order requires bidirectional filtering.

Comparison Table

Aspect	Single Direction	Both Directions
Filter Flow	One side only (Dimension to Fact)	Both sides mutually
Performance	Better — simpler query plan	Heavier — more complex DAX evaluation
Risk	Low — predictable filtering	Higher — unexpected filter loops possible
Best For	Standard Star Schema models	Many-to-many, bridge tables

Best Practice: Use Single direction by default. Enable Both directions only when specifically required, as it increases query complexity and risks ambiguous filtering.

Q10. Analyze the impact of improper data relationships on report accuracy.

Introduction

Data relationships are the structural backbone of the Power BI data model. When relationships are incorrectly defined — or missing — every DAX calculation and visualisation built on top of them will produce inaccurate or misleading results.

Impacts of Improper Relationships

- **Incorrect Aggregations:** A Sales total calculated without a proper relationship to the Date table will return a grand total in every row instead of a time-period-specific figure. For example, Monthly Revenue shows the same number for every month.

- **Double-Counting:** A Many-to-Many relationship without a bridge table causes the same fact row to be counted multiple times, inflating sales figures or headcount counts.
- **Missing Data in Visuals:** If a relationship is defined on the wrong column (e.g., ProductName instead of ProductID), products with the same name but different IDs appear merged or missing in reports.
- **Broken DAX Measures:** Functions like RELATED(), RELATEDTABLE(), and CALCULATE() rely on defined relationships to traverse tables. Without a relationship, these functions return BLANK or errors.
- **Incorrect Filtering:** A missing relationship between a slicer table and a fact table means the slicer has no effect — users believe they are filtering data, but all records remain visible.
- **Ambiguous Relationships:** Multiple active relationships between two tables cause DAX to be unable to determine which path to use, resulting in errors or incorrect values.

Example

Scenario: The Sales table and the Region table are not connected. A bar chart of 'Total Sales by Region' will either show all sales under a blank region label or fail to display any data — both misleading to business stakeholders making decisions on regional performance.

Conclusion: Relationship errors are the most dangerous type of error in Power BI because they produce plausible-looking but factually wrong reports. Always validate relationships in Model View and test measures against known data before publishing.

Q11. Explain the role of Query Editor in handling duplicate and null values.

Why Duplicates and Nulls Are Problematic

Duplicate records cause inflated counts and incorrect totals. Null values disrupt aggregations, prevent joins, and create blank entries in charts. Both issues must be resolved in the Power Query Editor before the data enters the model.

Handling Duplicate Values

- **Remove Duplicates (Entire Row):** Select all columns > Right-click > Remove Duplicates. Removes rows that are completely identical across all selected columns.
- **Remove Duplicates (Key Column):** Select the primary key column (e.g., OrderID) > Right-click > Remove Duplicates. Keeps the first occurrence and deletes subsequent duplicates on that key.
- **Group By:** Use Transform > Group By to aggregate duplicate rows into summary rows — e.g., grouping multiple order lines per customer into a single total per customer.

Handling Null / Missing Values

- **Remove Rows with Nulls:** Home > Remove Rows > Remove Blank Rows removes any row where the selected column contains a null or empty value.
- **Replace Values:** Right-click a column > Replace Values > Enter null in the 'Value to Find' field and provide a default replacement value (e.g., 0 for numeric columns, 'Unknown' for text).
- **Fill Down / Fill Up:** Transform > Fill > Down replaces nulls in a column by copying the non-null value from the cell above. Used for sparse datasets where a value applies to multiple rows below.
- **Conditional Column:** Add Column > Conditional Column creates a new column with a rule — e.g., if CustomerID is null, assign 'Walk-in Customer'.

Applied Steps Visibility

Each null-handling or duplicate-removal action is recorded as a named step in the Applied Steps pane, making the process fully auditable and reversible.

From Notes: 'Data cleaning ensures the dataset is accurate, consistent, and error-free.' Handling duplicates and nulls is the first and most critical cleaning task in any data preparation workflow.

Q12. Illustrate the process of creating and editing relationships between tables in Power BI.

Overview

In Power BI, relationships link tables through common key columns. They allow the Power BI engine to filter and aggregate data across tables, enabling multi-table analysis in a single report.

Creating a Relationship — Method 1: Drag and Drop

22. **Open Model View:** Click the Model View icon on the left sidebar.
23. **Identify Common Columns:** Locate the column that is shared between two tables — e.g., ProductID in both Dim_Product and Fact_Sales.
24. **Drag the Column:** Click and drag ProductID from Dim_Product and drop it onto ProductID in Fact_Sales. A relationship line appears connecting the two tables.
25. **Verify Cardinality:** Power BI automatically assigns cardinality. Check the '1' and '*' markers. '1' = unique values (Dimension), '*' = repeated values (Fact).

Creating a Relationship — Method 2: Manage Relationships

26. **Go to Home > Manage Relationships:** Opens a dialog listing all existing relationships.
27. **Click 'New':** Opens the Create Relationship dialog.
28. **Select Tables and Columns:** From the dropdowns, select the two tables and the matching columns.
29. **Set Properties:** Choose Cardinality (One-to-Many, Many-to-Many, One-to-One) and Cross-filter Direction (Single or Both).
30. **Click OK:** The relationship is created and appears in the Model View.

Editing an Existing Relationship

- **Double-click the relationship line:** Opens the Edit Relationship dialog.
- **Modify cardinality or cross-filter direction:** Change as required and click OK.
- **Alternatively:** Home > Manage Relationships > Select the relationship > Edit.

Example

Dim_Date (Date column, One side) linked to Fact_Sales (OrderDate column, Many side) with Single cross-filter direction. This allows time-based slicing of all sales data.

Best Practice: Always use surrogate integer keys (e.g., DateKey = 20240115) rather than text or date columns as relationship keys for optimal performance.

Q13. Demonstrate how to perform data cleaning operations in Power BI Desktop.

Opening Power Query Editor

Data cleaning in Power BI is performed entirely in the Power Query Editor. Access it via Home > Transform Data in Power BI Desktop. All cleaning steps are recorded in the Applied Steps pane.

Core Data Cleaning Operations with Steps

- **Step 1 – Remove Duplicate Rows:** Select the key column (e.g., CustomerID) > Right-click > Remove Duplicates. Result: each customer ID appears only once.
- **Step 2 – Remove Blank Rows:** Home > Remove Rows > Remove Blank Rows. Removes rows where all selected columns are empty.
- **Step 3 – Fix Data Types:** Click the data type icon on any column header. Change Text to Whole Number, Decimal, or Date as appropriate. Incorrect data types cause DAX errors.
- **Step 4 – Handle Null Values:** Select a column with nulls > Right-click > Replace Values > Value to Find: null > Replace with: 0 (for numbers) or 'N/A' (for text).
- **Step 5 – Trim Extra Spaces:** Select a text column > Transform tab > Format > Trim. Removes leading and trailing spaces that cause grouping errors (e.g., ' Mumbai' treated differently from 'Mumbai').
- **Step 6 – Standardise Text Case:** Transform tab > Format > UPPERCASE, lowercase, or Capitalize Each Word. Ensures consistent grouping in charts.
- **Step 7 – Replace Incorrect Values:** Transform > Replace Values. Replace known errors — e.g., 'N.A.' with 'Not Available', or fix abbreviations.
- **Step 8 – Remove Unnecessary Columns:** Select columns not needed for analysis > Right-click > Remove Columns. Reduces model size and improves performance.
- **Step 9 – Filter Irrelevant Rows:** Click the dropdown arrow on a column > Uncheck values to exclude test data, cancelled records, or future-dated rows.
- **Step 10 – Close & Apply:** Home > Close & Apply. The cleaned data is loaded into the Power BI model.

From Notes: 'Data cleaning techniques include Remove duplicates, Handle missing values, Correct spelling errors, Standardise formats, Remove blank rows, Fix incorrect data types.'

Q14. Explain the importance of measures in Power BI report generation.

What is a Measure?

A measure is a dynamic calculation written in DAX (Data Analysis Expressions) that is evaluated at query time based on the current filter context of a report. Unlike calculated columns (which compute a static value for each row), measures compute their results based on whatever slicers, filters, or visual context are applied at the time of viewing.

Syntax

Total Sales = SUM(Fact_Sales[SalesAmount])

Profit Margin = DIVIDE([Total Profit], [Total Revenue], 0)

Why Measures are Important

- **Dynamic Calculations:** Measures automatically recalculate when a slicer or filter is applied. The same 'Total Sales' measure returns Q1 sales when Q1 is selected, and Q3 sales when Q3 is selected — without any code change.
- **Report Accuracy:** Measures written with correct DAX logic ensure consistent, accurate calculations across every visual in the report. Business logic is defined once and reused everywhere.

- **KPI Tracking:** Measures are used to create KPIs such as Total Revenue, Profit Margin %, Year-over-Year Growth, and Customer Retention Rate — the core metrics of any business dashboard.
- **Flexibility:** A single measure can be placed in bar charts, tables, cards, and line charts simultaneously, displaying the same correct value across all visuals.
- **No Data Storage Overhead:** Unlike calculated columns, measures do not store computed values in the model. They are computed on-the-fly, keeping the model size smaller.

Measures vs. Calculated Columns

Aspect	Measure	Calculated Column
Evaluated	At query time (dynamic)	At data refresh time (static per row)
Filter Aware	Yes — responds to slicers and filters	No — fixed value per row
Storage	No storage used	Stored in the model table
Use For	KPIs, totals, ratios, aggregations	Row-level classification, flags, text combinations

Key Point: Every meaningful number displayed in a Power BI report — revenue, margin, count, growth — is delivered by a DAX measure. Without well-written measures, reports show raw data rather than business insights.

Q15. Analyze the role of data modeling in improving report performance.

What is Data Modeling?

Data modeling in Power BI is the process of defining the structure of imported tables, the relationships between them, and the calculations applied to them. A well-designed data model is the single biggest factor that determines report speed and analytical correctness.

How Data Modeling Improves Performance

- **Star Schema Reduces Query Complexity:** A Star Schema with one central Fact table and surrounding Dimension tables minimises the number of joins required to answer a query. Fewer joins = faster execution.
- **VertiPaq Compression:** Power BI's in-memory VertiPaq engine compresses data column-by-column. Dimension tables with low cardinality (few unique values) compress extremely well, dramatically reducing memory usage.
- **Fewer Unnecessary Columns:** A well-modeled dataset includes only required columns. Removing unused columns during data modeling directly reduces the in-memory footprint and speeds up refresh and query time.
- **Correct Data Types:** Integer keys in relationships are processed faster than text keys. Setting the correct data type on every column allows the VertiPaq engine to optimise storage and retrieval.
- **Aggregation Tables:** For very large datasets, pre-aggregated summary tables can be added to the model. Power BI automatically uses the summary table for high-level queries and falls back to detail data only when required.
- **DAX Efficiency:** A clean model allows DAX measures to traverse relationships efficiently. Complex DAX over a poorly modeled flat table runs significantly slower than the same logic over a Star Schema.

Good vs. Poor Data Model Impact

Scenario	Poor Model (Flat Table)	Good Model (Star Schema)
10M row report load	30-60 seconds	2-5 seconds
DAX execution	Slow, complex calculations	Fast, optimised traversal
File size	Large .pbix file	Significantly smaller

Conclusion: Data modeling is not an optional step. A correctly designed Star Schema model transforms a slow, unresponsive report into a fast, reliable analytical tool.

Q16. Compare calculated columns and measures with suitable examples.

Overview

Both calculated columns and measures are created using DAX in Power BI. However, they serve fundamentally different purposes and behave differently in terms of when they are evaluated and how they interact with filters.

Calculated Columns

- **Definition:** A calculated column is a new column added to a table using a DAX formula. It is evaluated row-by-row during data refresh and the computed value is stored in the model for every row.
- **Syntax Example:** Profit = Fact_Sales[Revenue] - Fact_Sales[Cost] | FullName = Customer[FirstName] & ' ' & Customer[LastName]
- **Key Characteristic:** The value is fixed for each row and does not change when filters or slicers are applied in the report.
- **Best Used For:** Row-level classification, text combinations, flags, and categories — e.g., classifying each order as 'High', 'Medium', or 'Low' based on order amount.

Measures

- **Definition:** A measure is a DAX formula that produces a single value calculated dynamically based on the current filter context of the visual or slicer.
- **Syntax Example:** Total Sales = SUM(Fact_Sales[Revenue]) | Avg Order Value = AVERAGE(Fact_Sales[OrderAmount])
- **Key Characteristic:** The value changes automatically when filters are applied — showing only the result relevant to the current selection.
- **Best Used For:** KPIs, totals, ratios, and percentages displayed in charts and cards.

Comparison Table

Aspect	Calculated Column	Measure
Evaluated	At data refresh — row by row	At query time — dynamically
Filter Context	Not filter-aware (static per row)	Fully filter-aware (responds to slicers)
Storage	Stored in model — uses memory	Not stored — computed on demand
Use in Visual	As a dimension/category/flag	As a value/metric/KPI
Example	Profit per row, Full Name column, Region flag	Total Revenue, YoY Growth %, Avg Sales

Rule of Thumb: If you need a value for every row in a table — use a calculated column. If you need an aggregated value that changes with filters — use a measure.

Q17. Explain how Power BI supports data transformation using Query Editor tools.

Overview

The Power Query Editor provides a comprehensive set of transformation tools accessible through the ribbon interface. These tools allow users to reshape, clean, and enrich data using a code-free, visual interface — while recording every step automatically.

Transformation Tools Available in Power Query Editor

Home Tab — Basic Operations

- **Remove Rows:** Remove top N rows, blank rows, duplicate rows, or rows with errors.
- **Keep Rows:** Keep top N rows or every other row — useful for extracting header data or sampling.
- **Sort Ascending / Descending:** Sort the table by any column for easier inspection.
- **Choose / Remove Columns:** Select which columns to keep or remove from the model.

Transform Tab — Column-Level Operations

- **Change Data Type:** Convert columns to the correct type: Whole Number, Decimal, Text, Date, True/False.
- **Split Column:** Divide a column by delimiter (comma, space, dash) or fixed width.
- **Merge Columns:** Combine multiple columns into one using a separator.
- **Format:** Apply text transformations: Trim (remove spaces), Clean (remove non-printable characters), UPPERCASE, lowercase, Capitalize Each Word.
- **Extract:** Pull specific portions of text — first N characters, last N characters, text between delimiters.
- **Replace Values:** Find a specific value and replace it with another across the entire column.

Add Column Tab — Creating New Columns

- **Custom Column:** Create a new column using a custom M language formula.
- **Conditional Column:** Create a new column based on if-then-else rules without writing code.
- **Index Column:** Add a sequential row number column — useful for creating unique keys.
- **Duplicate Column:** Copy an existing column before applying destructive transformations.

Applied Steps Pane

Every action is recorded as a named step. Steps can be renamed, reordered by drag-and-drop, or deleted. This makes the entire transformation pipeline transparent, auditable, and reversible.

From PPT: 'Each action is recorded in the Applied Steps pane, making the process transparent and reproducible. The M language enhances how Power BI transforms data, facilitating customisation beyond the standard interface.'

Q18. Demonstrate the process of combining data from multiple sources in Power BI Desktop.

Why Combine Data?

Business data is typically spread across multiple systems — sales in a SQL database, HR data in Excel, marketing data from Google Analytics. Combining these sources in Power BI creates a unified analytical model that supports cross-functional insights.

Method 1: Merge Queries (SQL JOIN equivalent)

31. **Open Power Query Editor:** Home > Transform Data.
32. **Select the primary table:** e.g., Fact_Sales.
33. **Click Home > Merge Queries:** Opens the Merge dialog.
34. **Select the second table:** e.g., Dim_Customer. Select the matching key column in both tables (CustomerID).
35. **Choose Join Type:** Left Outer (keep all rows from left), Inner (only matching rows), Full Outer (all rows from both).
36. **Expand the Merged Column:** A new column appears. Click the expand icon to select which columns from Dim_Customer to bring into Fact_Sales.
37. **Close & Apply:** The merged result is loaded into the model.

Method 2: Append Queries (SQL UNION equivalent)

38. **Load both tables:** e.g., Sales_2023 and Sales_2024 — both with the same column structure.
39. **Select Sales_2023 > Home > Append Queries:** Choose whether to append to the current table or create a new one.
40. **Select Sales_2024 as the table to append:** Rows from Sales_2024 are added below Sales_2023, creating a combined multi-year dataset.

Method 3: Data Model Relationships (recommended for Star Schema)

Rather than merging tables in Power Query (which denormalises data), the recommended approach for large datasets is to keep tables separate and define relationships in the Model View. This preserves the Star Schema structure and delivers better performance.

From PPT: 'Insights grow when data from different systems is brought together. Integrating CRM, ERP, marketing, or finance sources in Power BI creates a unified view of performance and eliminates siloed reporting.'

Q19. Analyze the significance of cross-filter direction in interactive visualizations.

What is Cross-Filter Direction?

Cross-filter direction controls how filter context propagates between related tables when a user interacts with a visual — such as clicking a bar in a chart, selecting a slicer value, or using a drill-down. It is a property of each individual relationship and significantly impacts how interactive reports behave.

Significance for Interactive Reports

- **Controls Slicer Behaviour:** In a report with a Region slicer linked to a Sales table via a Single-direction relationship, selecting 'South' in the slicer filters the Sales visual. With Both directions, selecting a specific sale in the table could also filter the Region slicer.
- **Enables Dynamic Filtering in Complex Models:** When a bridge table connects two fact tables (e.g., SalesRep table bridging Fact_Sales and Fact_Targets), Both directions allow a SalesRep slicer to filter both fact tables simultaneously.

- **Affects Measure Accuracy:** Some DAX measures produce incorrect results when Both directions is enabled on all relationships due to ambiguous filter paths. This is why Single direction is the safer default.
- **Cross-Highlighting vs. Cross-Filtering:** When clicking a bar in a chart in Power BI, related visuals can either cross-filter (remove unrelated data) or cross-highlight (dim unrelated data). This behaviour is also influenced by the filter direction settings.

Practical Example

A report has a Customer table (Dim) and Sales table (Fact) with a Single direction relationship. A chart shows Sales by Customer. If the user clicks one customer, only that customer's sales bar is highlighted — the filter flows correctly from Dim to Fact. With Both directions, the Customer slicer on the report side would also be filtered by whatever is selected in the Sales visual, providing a more tightly integrated interactive experience.

Best Practice: Default to Single direction for all relationships in a standard Star Schema. Enable Both directions only for bridge tables or specific interactive requirements, and always test for unintended filter behaviour after enabling it.

Q20. Explain how relationships between tables improve analytical reporting.

Introduction

Relationships between tables are the mechanism that makes multi-table analysis possible in Power BI. Without relationships, each table is an isolated island of data. With relationships, the entire model becomes a connected analytical network where filters and calculations flow seamlessly between tables.

How Relationships Improve Analytical Reporting

- **Enables Cross-Table Analysis:** A relationship between Fact_Sales and Dim_Customer allows a single report to show 'Total Revenue by Customer City' — combining the revenue figure from the Fact table with the City attribute from the Dimension table.
- **Eliminates Data Duplication:** Without relationships, the customer's city and name would need to be repeated in every row of the Sales table. Relationships allow attributes to live in one normalised table and be reused across the entire model.
- **Supports Time Intelligence:** A relationship between the Sales table and a Date Dimension table enables powerful DAX time-intelligence functions: Year-to-Date Revenue, Month-over-Month Growth, Same Period Last Year comparisons.
- **Enables Drill-Down Hierarchies:** Relationships support hierarchies — e.g., Year > Quarter > Month > Day. A single date relationship enables users to drill from yearly totals all the way down to daily sales in one click.
- **Consistent Filter Propagation:** When a user selects 'Q1 2024' in a Date slicer, the filter propagates through the Date relationship to filter every connected visual simultaneously — providing a consistent, synchronised report experience.
- **Supports Role-Playing Dimensions:** A single Date table can serve multiple roles — Order Date, Ship Date, Delivery Date — by creating multiple relationships (one active, others inactive) and using USERELATIONSHIP() in DAX.

Key Insight: Relationships transform a collection of separate tables into a unified analytical model. They are what allow Power BI to answer complex business questions like 'What was

the revenue from repeat customers in Q3 by product category?' from a single dashboard interaction.

Q21. Design a workflow for transforming raw business data into meaningful insights using Power BI.

Overview

The following is a complete, industry-standard workflow for converting raw business data into actionable visual insights using Power BI Desktop and Power BI Service.

Phase 1: Data Discovery and Connection

41. **Identify Data Sources:** List all relevant sources: SQL Server (sales), Excel (HR), Google Analytics (web), CRM (customers).
42. **Connect via Get Data:** Home > Get Data > select each source type. Use the Navigator window to select only required tables.

Phase 2: Data Transformation (Power Query Editor)

43. **Open Power Query Editor:** Home > Transform Data.
44. **Fix Data Types:** Ensure all columns have correct types: dates as Date, amounts as Decimal, IDs as Whole Number.
45. **Clean Data:** Remove duplicates, handle nulls (Replace/Fill Down), trim spaces, standardise text casing.
46. **Shape Data:** Split or merge columns, unpivot wide tables, filter irrelevant rows, remove unnecessary columns.
47. **Combine Data:** Use Merge Queries (join) or Append Queries (union) to consolidate data from multiple sources.
48. **Close & Apply:** Load clean, shaped data into the Power BI model.

Phase 3: Data Modelling

49. **Build Star Schema:** Confirm Fact tables (transactional) and Dimension tables (descriptive) are correctly structured.
50. **Define Relationships:** In Model View, create relationships between Fact and Dimension tables on matching key columns.
51. **Set Cross-Filter Direction:** Default to Single direction. Enable Both only where required.
52. **Create Measures:** Write DAX measures for KPIs: Total Revenue, Growth %, Avg Order Value.

Phase 4: Report Design

53. **Design Report Pages:** Add pages for different audiences: Executive Summary, Sales Detail, Regional Analysis.
54. **Add Visuals:** Drag fields onto the canvas. Choose appropriate visual types: KPI cards for totals, bar charts for comparisons, line charts for trends, maps for geographic data.
55. **Add Interactivity:** Insert slicers for Date, Region, and Product Category. Configure visual interactions.

Phase 5: Publishing and Sharing

56. **Publish to Power BI Service:** Home > Publish > select workspace.
57. **Create Dashboard:** Pin key tiles from reports to a Dashboard for executive overview.
58. **Set Up Scheduled Refresh:** Configure automatic data refresh daily or hourly.

59. **Share with Stakeholders:** Share via workspace, email link, or embed in Teams/SharePoint.

End Result: Stakeholders access a real-time, interactive dashboard that answers business questions in seconds — without any manual reporting effort.

Q22. Analyze the importance of saving and managing Power BI work files efficiently.

Power BI File Formats

Format	Extension	Contents	Use Case
Power BI Report	.pbix	Data model, queries, visuals, and actual data	Standard working file for development
Power BI Template	.pbix	Report structure and queries — no actual data	Reusable report layout for multiple datasets
Power BI Dataflow	Cloud	Reusable transformation logic in Power BI Service	Shared ETL across multiple reports

Why Efficient File Management Matters

- **Version Control:** Saving regular versions of the .pbix file (e.g., SalesReport_v1.pbix, SalesReport_v2.pbix) prevents accidental loss of working states. This is critical when testing new transformations or DAX measures.
- **Template Reusability (.pbix):** Saving a report as a .pbix template preserves all transformation steps, data model relationships, measures, and visual layouts — but not the actual data. Any analyst can apply this template to a new dataset instantly.
- **Backup Strategy:** Storing .pbix files in a version-controlled repository (SharePoint, OneDrive, or Git) protects against accidental overwrites and enables rollback to previous versions.
- **File Size Management:** Large .pbix files slow down Power BI Desktop and publishing. Efficient management includes: removing unused tables and columns, disabling 'Load' for staging queries, and using incremental refresh for large datasets.
- **Workspace Organisation:** In Power BI Service, organising published reports into dedicated workspaces (by department or project) prevents confusion and supports role-level access management.
- **Scheduled Refresh Management:** Published datasets in Power BI Service require gateway configuration and refresh schedules. Poor management leads to stale data being shown to decision-makers.

From Notes: 'PBIX is the default Power BI report file containing data, model, and visuals. PBIX is a Power BI template file storing report structure but not the actual data.'

Q23. Explain how Power BI Desktop supports data preparation for dashboard creation.

Role of Power BI Desktop in Dashboard Creation

Dashboards in Power BI can only be created in Power BI Service — but all the underlying data preparation that makes a dashboard accurate, fast, and meaningful is done entirely in Power BI Desktop. Desktop is the preparation engine; Service is the display platform.

How Desktop Supports Dashboard Creation

- **Data Connection:** Desktop connects to all data sources (SQL, Excel, APIs) and loads them into the model. Without this foundation, there is no data for dashboards to display.
- **Data Cleaning (Power Query):** Raw data from source systems contains errors, nulls, and inconsistencies. Desktop's Power Query Editor cleans and transforms the data so that every KPI pinned to a dashboard shows accurate figures.
- **Data Modelling:** Desktop establishes relationships between tables, ensuring that a 'Total Revenue by Region' tile on a dashboard correctly links the Sales and Region tables. A broken model produces wrong dashboard values.
- **DAX Measures for KPIs:** Every KPI tile on a dashboard — Total Revenue, Customer Count, Profit Margin — is powered by a DAX measure created in Desktop. Without measures, dashboards can only display raw fields, not calculated business metrics.
- **Report Design:** Desktop's Report View is where individual pages are designed. Dashboard tiles are created by pinning visuals from these report pages. The quality and relevance of report visuals determines the usefulness of the dashboard.
- **Optimisation:** Desktop allows testing report performance before publishing. Slow reports result in slow-loading dashboards. Performance tuning (removing unused columns, correct data types, aggregations) is done in Desktop.

Workflow

Power BI Desktop (Connect > Clean > Model > Report > Publish) -> Power BI Service (Dashboard creation by pinning tiles from published reports) -> End Users (Access dashboard via browser or mobile app)

Key Insight: Every element visible on a Power BI dashboard traces back to a decision made in Power BI Desktop — the data source, the transformation, the relationship, or the measure. Desktop quality = Dashboard quality.

Q24. Demonstrate the use of Query Editor for shaping and restructuring data.

What is Data Shaping in Power Query?

Data shaping refers to changing the structure of data so it fits the required format for analysis — reorganising columns, transposing tables, pivoting/unpivoting, and merging or appending queries. Power Query Editor provides all these tools through its ribbon interface without writing code.

Demonstration: Shaping a Wide Sales Table

Scenario: A raw Excel file has separate columns for each month's sales (Jan, Feb, Mar ... Dec). Power BI requires a tall format with one row per month for proper time-series analysis.

60. **Load the file:** Get Data > Excel > Navigator > Select the Sales sheet > Transform Data.
61. **Select the month columns:** Hold Ctrl and click Jan, Feb, Mar through Dec to select all 12 month columns.
62. **Unpivot Selected Columns:** Transform tab > Unpivot Columns > Unpivot Selected Columns. The 12 columns collapse into two columns: 'Attribute' (month name) and 'Value' (sales amount).
63. **Rename Columns:** Right-click 'Attribute' column > Rename > type 'Month'. Right-click 'Value' column > Rename > type 'Sales Amount'.

64. **Convert Data Type:** Select 'Sales Amount' > change data type to Decimal Number. Select 'Month' > change data type to Text.

Other Common Shaping Operations

- **Transpose:** Transform > Transpose. Rotates the entire table — rows become columns and columns become rows. Useful when source data is structured in the opposite orientation.
- **Merge Queries (Join):** Home > Merge Queries. Combines columns from a related table using a matching key — equivalent to a SQL LEFT JOIN.
- **Append Queries (Union):** Home > Append Queries. Stacks rows from two tables with the same structure — equivalent to a SQL UNION ALL.
- **Pivot Column:** Transform > Pivot Column. Converts unique values in a column into separate columns — opposite of Unpivot.
- **Group By:** Transform > Group By. Aggregates rows — e.g., total sales per customer, count of orders per region.

From PPT: 'Data reshaping: Exports rarely match the structure needed for analysis. Use tools like Transpose to convert rows into columns, or Unpivot Columns to normalise nested data.'

Q25. Propose a real-world scenario where data transformation and relationship management are essential in Power BI.

Scenario: National Retail Chain — SmartMart

Company Background

- **Operations:** SmartMart has 200 stores across 5 regions, 50,000 SKUs, 3 million customer transactions monthly.
- **Current Challenge:** Sales data is in SQL Server, inventory data is in a separate Oracle ERP, customer loyalty data is in an Excel file updated weekly by the marketing team, and HR staffing data is in a CSV export from the HR system. No integrated view exists.
- **Business Problem:** Regional managers cannot see how staffing levels affect sales performance, or how inventory shortages relate to revenue loss — because data lives in isolated systems.

Data Transformation Requirements

- **SQL Server Sales Data:** Contains nulls in 'Discount Applied' column, dates stored as text strings, duplicate transaction IDs from system migration. Requires: Fix date type, remove duplicates, replace null discounts with 0.
- **Oracle Inventory Data:** Product names use abbreviated codes ('Elec' instead of 'Electronics'). Requires: Replace values to standardise category names, merge ProductCode with Dim_Product table.
- **Excel Loyalty Data:** Customer names in mixed casing, some customer IDs missing, data in wide format (one column per month for loyalty points). Requires: Standardise casing, fill down IDs, unpivot to tall format.
- **HR CSV Data:** Store codes use different format from Sales table (STORE001 vs S001). Requires: Custom column using Text.Replace() to standardise store code format before creating relationship.

Relationship Management

- **Fact_Sales to Dim_Product:** Many-to-One on ProductID. Single cross-filter direction.
- **Fact_Sales to Dim_Customer:** Many-to-One on CustomerID. Single cross-filter direction.

- **Fact_Sales to Dim_Date:** Many-to-One on OrderDateKey. Single cross-filter direction. Enables time intelligence.
- **Fact_Inventory to Dim_Product:** Shared Dim_Product enables cross-analysis between sales and inventory.
- **Dim_Store to Fact_Sales:** Many-to-One on StoreCode. After transformation, store codes now match.

Expected Business Outcomes

Business Question	Answer Enabled by Transformation and Relationships
Which stores are understaffed relative to sales volume?	HR + Sales relationship after store code transformation
Which products are out of stock during peak demand?	Inventory + Sales relationship after product code standardisation
What is the loyalty redemption rate by region this quarter?	Customer loyalty (after unpivot) + Sales + Date relationship
YoY revenue growth by store category?	Date relationship enabling time-intelligence DAX measures

Conclusion: In this scenario, data transformation (cleaning, shaping, standardising) and relationship management (correct joins, cross-filter directions) are not optional — they are what make integrated reporting possible.

Q26. Apply Transformation steps for preparing HR datasets.

HR Dataset — Typical Issues

HR datasets exported from systems like SAP, Workday, or Excel typically contain: combined name fields, inconsistent department codes, dates stored as text, missing employee IDs, duplicate records from rehiring, and salary data formatted as currency strings.

Sample Raw HR Data Structure

EmpID	Full Name	DOJ	Department	Salary
101	Priya sharma	12-01-2022	HR	\$45,000
	AMIT VERMA	15/03/2021	hr	\$52000
103	Priya sharma	12-01-2022	HR	\$45,000

Transformation Steps in Power Query Editor

65. **Fix Employee ID nulls:** Select EmpID column > Add Column > Index Column (starting from 101). This assigns unique IDs to rows with missing Employee IDs. Alternatively: Fill Down or Replace with 'PENDING'.
66. **Standardise Name Casing:** Select 'Full Name' column > Transform tab > Format > Capitalize Each Word. 'priya sharma' becomes 'Priya Sharma'; 'AMIT VERMA' becomes 'Amit Verma'.
67. **Split Full Name:** Select 'Full Name' column > Transform > Split Column > By Delimiter (Space) > Split at first delimiter. Creates 'First Name' and 'Last Name' columns. Rename both.

68. **Fix Date of Joining Format:** Select 'DOJ' column > Change Data Type > Date. Power BI automatically handles multiple date formats (12-01-2022 and 15/03/2021) during conversion. Verify in Data View.
69. **Standardise Department Codes:** Select 'Department' column > Transform > Format > UPPERCASE or Capitalize Each Word. 'hr' and 'HR' are now both 'Hr'. Then use Replace Values: replace 'Hr' with 'Human Resources'.
70. **Clean Salary Column:** Select 'Salary' column > Transform > Replace Values: replace '\$' with "" and ',' with ". Then change Data Type to Decimal Number. '\$45,000' becomes 45000.
71. **Remove Duplicate Records:** Select 'Full Name' and 'DOJ' columns together > Right-click > Remove Duplicates. Removes the duplicate Priya Sharma entry.
72. **Add Derived Columns:** Add Column > Custom Column: Years of Service = `DateTime.LocalNow()[Year] - Date.Year([DOJ])`. Adds current tenure for each employee.
73. **Filter Active Employees Only:** If a 'Status' column exists: filter to keep only rows where Status = 'Active'.
74. **Close & Apply:** Home > Close & Apply. Clean HR data is loaded into the Power BI model ready for visualisation.

Outcome: After transformation, the HR dataset is clean, consistently formatted, and ready for reports like Department Headcount, Average Tenure by Department, Salary Distribution, and Joining Trend Analysis.

Q27. Demonstrate one-to-many relationship in Power BI.

What is a One-to-Many Relationship?

A One-to-Many relationship is the most common and recommended relationship type in Power BI. It connects a Dimension table (the 'One' side, containing unique values) to a Fact table (the 'Many' side, containing repeated references to those values). The Dimension table has one unique row for each key value, while the Fact table can have many rows referencing the same key.

Real-World Example

Scenario: A retail company has two tables — Dim_Customer (one row per customer) and Fact_Orders (multiple orders per customer).

Dim_Customer (One Side)	Fact_Orders (Many Side)
CustomerID: 1001 Name: Priya City: Mumbai	OrderID: 5001 CustomerID: 1001 Amount: 2000
CustomerID: 1002 Name: Rahul City: Pune	OrderID: 5002 CustomerID: 1001 Amount: 3500
CustomerID: 1003 Name: Sneha City: Delhi	OrderID: 5003 CustomerID: 1002 Amount: 1200
	OrderID: 5004 CustomerID: 1001 Amount: 800

CustomerID appears once in Dim_Customer (One side) but appears multiple times in Fact_Orders (Many side) — Customer 1001 has three orders.

Steps to Create the Relationship in Power BI

75. **Load both tables:** Import Dim_Customer and Fact_Orders into Power BI Desktop via Get Data.

76. **Open Model View:** Click the Model View icon on the left sidebar.
77. **Create Relationship:** Drag CustomerID from Dim_Customer and drop it onto CustomerID in Fact_Orders. A relationship line appears.
78. **Verify Cardinality:** The relationship line shows '1' on the Dim_Customer side and '*' (asterisk) on the Fact_Orders side — confirming One-to-Many.
79. **Set Cross-Filter Direction:** Default is Single (Dim_Customer filters Fact_Orders). This is correct for standard reporting.

How It Works in a Report

- **Filter Propagation:** When a user selects 'Mumbai' in a City slicer, Power BI uses the One-to-Many relationship to filter Fact_Orders to show only orders from Mumbai customers — even though 'City' is in Dim_Customer and 'Amount' is in Fact_Orders.
- **DAX Interaction:** Total Orders by City = CALCULATE(COUNT(Fact_Orders[OrderID]), RELATED(Dim_Customer[City])) — this works correctly only because of the One-to-Many relationship.
- **Result:** A bar chart of 'Total Revenue by City' correctly aggregates order amounts from Fact_Orders grouped by City from Dim_Customer.

Key Rule: In a One-to-Many relationship, the column on the 'One' side (Dimension) must contain unique, non-null values. If duplicates exist in the Dimension key column, Power BI will either create a Many-to-Many relationship or show a warning.